AIR UNIVERSITY ISLAMABAD CAMPUS
ADVANCED DATABASE MANAGEMENT SYSTEMS LAB (CS-330L)
SPRING 2026
PROJECT
CLO- 2 GA-5

Due Date: June 01, 2026 (01:59 PM)

Marks:100

Instructions:

1. *Make sure that you read and understand each and every instruction.*
2. *Submit a single '.zip' file for your project that will have all the required files for your project.*
3. ***Rename the submission with the roll number of each member of your group.***

NovaMart Group: A Disastrous Case Study

A Note Before You Begin

What follows is a case study, not a problem set. There is no numbered list of tasks. There is no section that tells you what is wrong with the system. There is no specification of what your solution should look like. The case describes a company, an architecture, a six-month operational history, and a current state of crisis. Inside that narrative are the failures you will diagnose, the constraints you will work within, and the decisions you will make and defend.

Your first deliverable - before any code is written - is your own list of problems. You will read this document, examine the data sample, the log files, the configuration exports, and the post-incident reports. You will identify what is wrong. You will prioritise. You will decide what your team will fix and what you will document as known issues that you chose not to fix and why. Two teams reading this case study could produce two different problem lists. That is intended. I will assess whether your list is correct, whether your prioritisation is defensible, and whether the things you chose not to fix are the right things to leave.

Once you have a list, the real work begins: for each problem, there are multiple technically defensible solutions; for each solution, there is a body of research literature with empirical findings that bear on whether the solution is right for this specific workload, this specific data distribution, this specific failure mode profile.

AI tools are expected and welcome. They will accelerate everything from log parsing to schema migration scripts to benchmark harnesses. They will not - in any case the instructors have seen - correctly identify what is wrong with a system from a narrative description, or correctly select between competing remediation strategies given workload-specific constraints, or correctly defend a decision against an evaluator who proposes an alternative. Those tasks belong to the humans on the team.

How to Read This Document

Read the entire case study before opening a single tool. The failures are interconnected - addressing one in isolation often unmasks or worsens another. The order in which you read the sections is not the order in which you should solve them. The order in which you solve them is one of the decisions the evaluator will probe.

How the Evaluator Will Read Your Submission

The evaluator does not have a check list of problems against which to grade your submission. The evaluator has read this case study, has independently identified what they consider to be the significant failures, and will compare your problem list to theirs. They will also examine the problems they did not identify but you did, and the problems they identified but you did not - both are scored. The case study contains more failures than any team is expected to solve in this time. Choosing what to address is part of the assessment.

Part One: The Company

NovaMart Group is a Pakistani conglomerate operating across four business verticals that share a common technology backbone but were built at different times by different teams under different leadership. NovaMart Online is a full-catalog e-commerce platform serving 14 million registered users with 2.1 million SKUs and daily transaction volumes between 80,000 and 220,000 orders depending on promotional events and seasonality. NovaMart Retail operates 134 physical stores across 23 cities, each with a point-of-sale system connected to the central database. NovaPay is a buy-now-pay-later and installment credit business with 2.3 million active credit accounts representing PKR 18 billion in outstanding portfolio. NovaLogistics manages last-mile delivery and seven regional warehouses, with its own fleet of 1,200 delivery vehicles and a small army of dispatch personnel coordinating through a single overworked dispatch application.

The four verticals were originally separate systems. Between 2014 and 2022 they ran on a single monolithic Oracle 19c installation that the original database administrators understood intimately. By the end of 2022, the executive team had concluded that this monolithic approach was a competitive liability - the language used in board minutes was ‘we cannot scale to compete with regional platforms while running on a single Oracle box’. In January 2023 the Chief Technology Officer commissioned a full-stack database modernisation under the banner of ‘cloud-native distributed data infrastructure’. The plan, as approved by the board, would move all four business verticals onto a heterogeneous architecture: PostgreSQL 16 as the primary online transaction processing system, MS SQL Server 2022 with SQL Server Analysis Services for business intelligence and analytics, Elasticsearch 8.x for product catalog search, Redis 7.x for application-level caching, and a Node.js middleware layer providing unified data access across all of these systems.

The migration was budgeted for 18 months. It was declared complete after 7 months. The pressure to declare completion came from investors who had been promised the new platform as a precondition for a Series C funding round; the funding round was scheduled, and the platform was declared ready in time for it. The CTO who approved the early declaration left the company three weeks later for an executive role elsewhere. The original database administration team - the four engineers who had operated the Oracle system since 2014 and who carried the institutional knowledge of how NovaMart's data actually behaved - resigned within thirty days of the declaration. None of them participated in a knowledge transfer process. None of them documented the operational practices they had developed over nine years. The replacement team was hired at lower cost, with junior profiles, and was told that the migration was complete and the system was production-ready.

The system was not production-ready. The first symptoms appeared within ten weeks. Customers complained about checkout failures, about cancelled orders that had already been charged, about prices in the search results that did not match prices on the product detail page, about loyalty rewards that were credited to the wrong account, about installment statements that did not match the customer's actual payment history. Each complaint was handled individually by customer support; no pattern was identified by management for months. By the time the operations team began consolidating the symptoms into an incident report, the system had been in a degraded state for nearly six months.

Three weeks ago, NovaMart suffered a partial ransomware event. An attacker gained access to one of the application servers through a vulnerability in an unrelated SaaS integration, escalated to the service user, read configuration files containing database credentials, connected to the primary database as a superuser, and exfiltrated

approximately 2.3 million customer records before deploying an encryption payload against a subset of the database files. The operations team responded by shutting down the Lahore data centre, which contained the only copies of certain data (a fact that was discovered during the response, not before it). Service was restored within seven hours through a combination of partial restoration from base backups and live operation on the Islamabad nodes only. The forensic investigation, conducted by an external cybersecurity firm, identified seven distinct security failures in the database infrastructure, any one of which would have been sufficient to enable the attack.

NovaMart is now under investigation by Pakistan's National Cyber Security Authority under the Prevention of Electronic Crimes Act 2016 and faces concurrent proceedings under the draft Personal Data Protection Bill 2023. The regulator has demanded a remediation report with technical evidence of corrective action within 45 days. The board meets in 31 days. Your team has been retained as emergency consultants.

Part Two: What You Have Been Given

Before describing the technical infrastructure, it is worth being precise about what you have access to. You have not been given administrative production access. You have not been given the ability to test changes against live data. What you have been given is the following, all delivered as files on a shared drive that was set up for your engagement:

- A 15 percent anonymised sample of production data, exported from all five PostgreSQL instances and the MS SQL Server warehouse. The sample preserves cardinality ratios and the distribution of edge cases. Customer identifiers, names, and contact details are replaced with synthetic values; transaction structure, foreign key relationships, and data shape are unchanged.
- All configuration files: postgresql.conf for all five PostgreSQL instances, the SSAS deployment configuration, the Redis configuration, the Elasticsearch index mappings, the Node.js middleware connection configuration, the load balancer routing rules, and the PgBouncer configuration (which is present but not currently deployed — the team that configured it never finished the deployment).
- The complete application source code: the Node.js middleware (4,800 lines), the customer-facing web application (Node.js + React, approximately 38,000 lines), the retail point-of-sale system (Java, approximately 14,000 lines), and the NovaPay back-end (Python, approximately 22,000 lines). You also have the source code of the data warehouse ETL pipeline (T-SQL stored procedures and an SSIS package).
- The full PostgreSQL slow query log from the primary OLTP instance, covering the past six months. It is 7.2 gigabytes uncompressed. Analysis of this log will reveal patterns that no individual query reveals.
- Snapshots of pg_stat_statements taken at five points in time: immediately after the migration declaration, three months later, immediately before the ransomware event, immediately after, and last week. These snapshots will tell you how the workload has evolved and which queries have degraded most severely.
- Forensic report from the ransomware event, prepared by the external cybersecurity firm. It documents the attack timeline, the evidence collected, and the security failures identified.
- All internal incident tickets from the past six months — approximately 1,400 of them — including customer support tickets, operations alerts, and post-incident reviews where they were conducted. Most were not.
- The reconciliation team's weekly reports for the past six months. These reports document discrepancies between the OLTP system, the data warehouse, and the financial books. The reports are extensive.
- Email threads between the CTO, the engineering leads, and outside consultants from the period of the original migration. These threads are unflattering. They are also informative — they reveal what decisions were made under what pressure and with what alternatives considered.
- A document titled 'NovaMart Disaster Recovery Plan v3' that was prepared during the migration and has never been tested. It specifies a 30-minute Recovery Time Objective and references a recovery procedure based on pg_dump.

Part Three: The Technical Landscape

The infrastructure you are inheriting spans two data centres, five PostgreSQL instances, one MS SQL Server installation, one Elasticsearch cluster, one Redis cluster, and a Node.js middleware layer that mediates access to all of these. Each component has its own configuration history, its own operational quirks, and its own contribution to the current crisis.

Physical Topology

Three PostgreSQL nodes operate in the Islamabad data centre. The first node, designated internally as Node 1, runs the primary online transaction processing workload — customer-facing reads and writes, order placement, payment processing. The second node, Node 2, holds parts of the customer and catalog tables — specifically, customers whose surname begins with letters A through M, and the master catalog data for physical goods. The third node, Node 3, holds inventory data and the logistics tables, including delivery routing and warehouse stock levels.

Two PostgreSQL nodes operate in the Lahore data centre. Node 4 was provisioned to hold NovaPay's installment credit data. Node 5 was provisioned as analytics staging for the warehouse extract-transform-load process. Both Lahore nodes were loaded with a full snapshot of their respective data sets approximately six months ago. They have not been synchronised with the Islamabad nodes since. The team responsible for setting up replication between the data centres encountered a TLS certificate problem during initial configuration, deferred the work, and never returned to it. The NovaPay application, which was originally intended to write to Node 4, has been writing instead to Node 1 since shortly after the migration was declared complete — a change made by an engineer who could not reach Node 4 from the application's network segment and routed the connection to Node 1 as a temporary fix. The temporary fix has been in production for six months.

There is no replication of any kind between any of the five PostgreSQL nodes. If any single node fails, the data on that node is unavailable until the node recovers. The Islamabad data centre and the Lahore data centre are connected by a private network link with a round-trip latency of 15 to 25 milliseconds depending on time of day and route conditions.

The MS SQL Server 2022 instance runs in the Islamabad data centre on dedicated hardware. It hosts the data warehouse and the SSAS multidimensional cube. The ETL pipeline that populates the warehouse runs nightly. The cube is consumed by 23 analyst workstations through Excel-based MDX queries; the queries were originally written against an earlier reporting system and were rewritten — incompletely — during the migration.

The Elasticsearch 8.x cluster consists of three nodes co-located with the PostgreSQL primary in Islamabad. It is intended to power product search on the customer-facing application. The Redis 7.x cluster is similarly co-located and is used for application-level caching of frequently accessed read operations. The Node.js middleware runs as a fleet of fifteen application servers behind a load balancer. Each application server maintains its own connection pool to the database tier.

Configuration State

The PostgreSQL configuration is the default postgresql.conf as installed. No tuning has been performed since the initial deployment. The primary OLTP instance runs on hardware specified as 128 gigabytes of RAM, 32 CPU cores, and NVMe solid-state storage with measured sustained throughput of 4 gigabytes per second write and 8 gigabytes per second read. The configuration sets shared_buffers to 128 megabytes, effective_cache_size to 4 gigabytes, work_mem to 4 megabytes, max_parallel_workers_per_gather to 2, max_connections to 100, and checkpoint_completion_target to 0.5. The wal_level is set to minimal. No WAL archiving is configured. The checkpoint_timeout is 30 minutes.

The application servers each run a connection pool configured for 50 connections. Fifteen servers therefore present a theoretical demand of 750 connections to a database that accepts 100. Under any moderate load, the connection

pool on individual application servers blocks waiting for available connections; under peak load, the block exceeds the application's HTTP request timeout of 30 seconds, producing cascading 503 errors to end users. The operations team's standard response is to restart application servers until the queue clears. This procedure takes between 8 and 12 minutes and produces additional order failures during the restart window. The runbook documents this procedure as the resolution for thirteen distinct alert types.

Eighteen months ago, autovacuum was disabled on four of the largest tables. The developer who made the change documented their reasoning in a Slack message: autovacuum was observed running during a performance incident, and the developer concluded that it was the cause of the incident. The actual cause of the incident has never been identified. The four affected tables — Products, Orders, OrderItems, and Customers — have not been manually vacuumed in eighteen months. Recent inspection of the tables shows that they have accumulated substantial bloat. Products occupies approximately 4.2 times its live row count in physical pages. OrderItems, which sees the highest update volume, occupies approximately 5.1 times its live row count. Orders is at 3.8 times and Customers at 2.4 times. Dead tuples occupy the difference.

Monitoring is limited to a server availability ping. Query latency percentiles are not collected. Connection pool utilisation is not collected. Buffer hit rate, lock wait events, autovacuum activity, checkpoint duration, replication lag (which is moot, since there is no replication, but which would be needed if there were), and I/O saturation are all uncollected. The only performance information available to the team is what users complain about. The team has been compensating by creating ad-hoc query log analyses in a series of one-off scripts, none of which have been productionised.

Part Four: The Data Itself

Some of what is wrong with NovaMart's system is structural — it is in the way the data is shaped, not in how the queries are written or how the server is configured. The 15 percent sample you have been given will let you examine the shape directly. What follows is what the case study authors observed when they examined the sample. Your team should verify and extend these observations.

The Products Table

The Products table has 47 columns. Thirty-nine of them are nullable. The table holds four fundamentally different kinds of things in the same set of rows. Physical goods — electronics, apparel, groceries, household items — are stored alongside digital goods (streaming subscriptions, software licenses, gift cards, mobile data top-ups), alongside financial products (NovaPay installment plan configurations, credit products), alongside logistics services (express delivery slots, warehouse storage agreements, last-mile service packages). A physical laptop row contains 24 NULL values where its columns are not applicable. A digital subscription row contains 33 NULL values. A NovaPay plan configuration row contains 38 NULL values.

The query planner's statistics for this table are computed across all four entity types. Cardinality estimates for type-specific predicates are wrong by factors of ten to a hundred. The planner's row count estimate for a typical product search predicate is 12 rows. The actual result set for the same predicate over a 42-million-row table is in the millions. The most recent ANALYZE on this table was run six months ago, immediately after the migration was declared complete. The table has grown by approximately 200,000 rows per month since then. It is must for you to design a report for the whole case scenario that you are provided. The report must highlight the problems that you have identified in the case study and your approach to solve those problems along with providing a justification for the problems that you hav echosen to solve.

Manual inspection of the table reveals that at least one NovaPay plan row stores a value in the column labelled weight_kg. Manual inspection further reveals that the description column, declared as VARCHAR(2000), contains text data for physical goods (multi-paragraph product descriptions), structured key-value text for digital goods (an ad-hoc serialisation that predates JSONB), and SQL-escaped JSON for some NovaPay plan rows (the result of an

unfinished migration script). The application code that reads this column does so in eleven distinct places, each making different assumptions about its format.

The Customers Table

The Customers table has 14 million rows. It contains the standard customer attributes — name, contact, address — plus a column called profile_data, declared as VARCHAR(8000), that was added during the migration. The intent was to hold customer profile information as JSON, replicating MongoDB-style document flexibility within the relational store. JSONB was not used; the column is a plain VARCHAR. The actual contents are inconsistent across rows.

Approximately 23 percent of rows contain malformed JSON. The malformation pattern suggests a bulk import script that truncated input strings at the column's 8000-character limit without first checking whether the truncation point fell inside a valid JSON structure. Closing brackets and quotation marks are missing in characteristic places. Of the 77 percent of rows that contain valid JSON, the structure is not consistent. Some rows contain a nested address object with sub-objects for district and province. Others contain flat address fields at the top level. A third group contains an array of multiple addresses, each with its own structure. A fourth group contains no address data at all and instead holds marketing preference flags that belong in a separate table according to the original schema design.

The eleven application code paths that parse this column do so using eleven different JSON parsing routines, each making different assumptions about which fields are present and how they are nested. The loyalty program mailer and the tax invoice generator have been sending mail to different addresses for the same customer for at least eight months — this was discovered when a customer who had moved cities received the loyalty mailer at the new address and the tax invoice at the old address in the same week and contacted customer support.

There is also a column called card_last_sixteen, of type VARCHAR(16). The column name is misleading. It contains the full sixteen-digit primary account number for 2.3 million customers who used a 'save full card' feature that NovaMart offered until 2021. The feature was removed from the user interface in 2021 because the engineering team realised it should not have been built. The data was not deleted. The forensic report from the ransomware event confirms that all 2.3 million PAN values were exfiltrated. Some of these records may be within an 18-month mandatory retention window for payment dispute resolution; blanket deletion is not legally permissible without classification first.

The Employees Table

The Employees table has 180,000 rows. NovaMart operates a six-tier management hierarchy: a small executive layer at the top (about 12 rows), regional directors below them (34 rows), area managers, store managers, department leads, and floor staff who form the bulk of the table. The hierarchy is encoded as a self-referential manager_id column. The column has no index.

Queries that retrieve the complete reporting chain for a specific employee — required for payroll authorisation escalation, performance review routing, and several other workflows — are written as recursive Common Table Expressions. They scan the entire 180,000-row table at every recursion level, with the maximum depth of the hierarchy being eight. A single such query takes between 8 and 14 seconds on the current hardware. The monthly payroll batch invokes this query 180,000 times. The payroll batch currently runs for 11 hours.

The NovaPay_Plans Table

The NovaPay_Plans table holds installment credit plans. It has 23 foreign key columns, of which 11 reference tables whose rows the foreign keys can no longer find. The foreign key constraints were disabled during the bulk data load phase of the migration to speed up loading. They were re-enabled for 12 of the 23 columns after loading completed. The remaining 11 were never re-enabled. The team intended to fix this in a follow-up sprint that never happened.

The application code that reads NovaPay_Plans wraps every query in a try-except block that silently absorbs foreign key violations. The query log records approximately 340,000 plan rows that have no corresponding customer, order, or product. These are orphans. They continue to exist. The nightly deduction batch attempts to process them; the deduction attempt fails (because the referenced customer no longer exists in the Customers table) and the deduction

batch logs the failure and moves on. Approximately 12 percent of the deduction batch's total runtime is spent on orphan plans. The deduction batch runs every night.

Some of the orphan plans represent real financial obligations. Specifically, a subset of them correspond to customers whose accounts were deleted as part of a 2023 data hygiene initiative but whose installment plans had not yet been fully paid off. These plans are technically still collectible under Pakistani consumer credit law; they cannot simply be deleted from the database without proper accounting treatment. The team that ran the data hygiene initiative was not aware that NovaPay had outstanding obligations against the accounts they deleted.

Part Five: How the System Behaves Under Load

The slow query log and the pg_stat_statements snapshots paint a consistent picture of a system whose query performance has degraded across virtually every dimension since the migration was declared complete. The degradation is not uniform. Some queries have become slightly slower; others have become catastrophically slow; a small number have become unable to complete at all under realistic load. The patterns matter, and the patterns are detectable only through analysis of the logs.

The Product Search Query

The product search query is the highest-traffic query in the system. It is invoked by every visitor who enters a search term on the customer-facing application, which is to say, by approximately 8 to 12 million distinct users per day. Its median latency, as measured from pg_stat_statements, is 47 seconds. Its 99th percentile is undefined because the application aborts the query after 30 seconds; the abortion triggers a retry, which is logged separately and which is itself slow. Under peak load, the retry cascade compounds and consumes most of the database's available resources.

An EXPLAIN ANALYZE on the query, taken on a Saturday morning at 8 AM before load builds, reveals a chain of unfortunate planner decisions. The Products table scan reports an estimated row count of 12 from a table of 42 million rows. The estimate is derived from statistics that have not been updated since the migration; the planner does not know that the table has grown. Believing it is processing 12 rows, the planner chooses sequential scan. Believing it is joining 12 rows to a 2-million-row Categories table, an 8.7-million-row Inventory table, a 340,000-row Suppliers table, and a 180-million-row Reviews table, the planner chooses nested loop joins. The Reviews join recomputes AVG(rating) GROUP BY product_id at query time over the full 180 million rows. The Inventory join is routed through a UNION ALL of 134 view definitions — one per retail store — that the planner does not recognise as a partitioning structure and therefore cannot prune.

Each of the five planner failures interacts with the others. Fixing the statistics alone reveals that the planner's second choice was a hash join with a hash table that would spill 2.4 gigabytes to disk because work_mem is set to 4 megabytes. Fixing work_mem alongside statistics reveals that the GIN index on the description column was built with the default operator class, which is appropriate for path queries but not for the containment queries the search is actually performing. Fixing the operator class reveals that the UNION ALL inventory join is now the bottleneck. Each fix unmasks the next. The query is not slow because of one mistake; it is slow because of five mistakes stacked on top of each other in a way that makes any single fix appear to do nothing.

The Order History Query

Customer service agents handling a support inquiry need to see a customer's recent order history. The query that retrieves it was written by a developer who believed that correlated subqueries and joins were equivalent in performance. The outer query returns customers matching a search criterion. For each customer in the outer query, the inner subquery scans the Orders table — 38 million rows — and the OrderItems table — 210 million rows. The application invokes this query in batches of 50 customers. Each batch triggers 50 full table scans of a 38-million-

row table and a 210-million-row table. A single batch consumes roughly 12.4 billion row-scans. The comment in the source code, beside the query, reads: ‘This could probably be optimised but it works fine’.

The Inventory Availability Check

Every retail transaction begins with an inventory availability check. The point-of-sale system queries the database to determine whether the items in the customer's basket are in stock at the current store. The check is routed through a UNION ALL of 134 view definitions — one per store, defined as SELECT * FROM inventory WHERE store_id = X. The views were created before PostgreSQL introduced declarative partitioning. The planner does not recognise them as a partitioning structure. Every availability check scans the full 8.7-million-row Inventory table and filters afterwards. The 99th percentile latency for this check is 12 seconds. A customer standing at a checkout counter waits 12 seconds while the database thinks. Onboarding a new store requires manually editing the view definition and redeploying the application; three months ago a new store in Sialkot operated without inventory visibility for four days because nobody updated the view.

The Slow Query Log as a Whole

The full slow query log contains 1,247 distinct query fingerprints. Analysis of the log — which is itself a non-trivial exercise; the log is 7.2 gigabytes — reveals that 67 of the 1,247 fingerprints account for 96 percent of the database's total CPU time. Of those 67, careful inspection shows that 12 are semantically equivalent to one another in groups: pairs and triples of queries that are written differently by different developers but compute the same logical result. Each variant produces a different execution plan; the database is doing the same work multiple ways. Identifying these equivalence classes is a known problem in the query analysis literature and requires a similarity metric rather than simple string comparison. The classes cannot be consolidated through application source code modification, because the application teams responsible for each variant are different and a coordinated change across all of them is not feasible within the project window.

Part Six: What the Incident Reports Reveal

The incident ticket archive — approximately 1,400 tickets from the past six months — contains the symptomatic record of failures that the operations team responded to without ever identifying their underlying causes. Read in sequence, the tickets reveal patterns. Read in conjunction with the source code, the patterns become diagnosable.

The Phantom Inventory Incidents

Eight hundred and forty-seven incidents in the past quarter were logged under the internal category 'inventory discrepancy.' Each incident describes a transaction in which a customer placed an order for an item that the inventory system reported as available, the payment was charged, and then — at fulfilment time — the item was discovered to be unavailable. In 134 of the 847 cases, NovaMart fulfilled both orders by drawing inventory from a different store's allocation and incurring express shipping costs to deliver to the originally promised location. In the remaining 713 cases, the customer received a cancellation email after their payment had been confirmed. The total direct support cost of these 713 cancellations last quarter was approximately 2.3 million Pakistani rupees, not including the indirect cost to the brand.

The pattern, when one examines the timestamps, is consistent. The incidents cluster on Friday evenings and during sale events — times of high concurrent checkout activity. The relevant section of the application source code, in the checkout module, performs a read of available inventory, makes a decision based on that read, and then writes a decrement in a subsequent statement. There is no isolation control around the read-decide-write sequence. The default isolation level applies. Two concurrent checkout sessions can both read the same available stock value, both decide independently that the purchase is allowable, and both decrement the stock — driving the inventory negative. The root cause has been attributed in three consecutive board reports to 'supply chain issues’.

The NovaPay Reconciliation Discrepancies

The financial reconciliation team produces a weekly report comparing the database value of customer.available_credit against the sum of installment-related transactions in the audit log. The two should agree. They do not agree for 2,847 customer accounts. The discrepancies range from PKR 500 to PKR 340,000 per account. The reconciliation team has been correcting these manually each week, using the audit log as the source of truth, at a cost of approximately 14 person-hours per week. The team's official explanation for the discrepancies is 'system instability.'

Examination of the source code reveals the pattern. Multiple application code paths update available_credit — installment payment received, new plan created, plan modified, plan paid off. Each update reads the current value, computes an adjustment, and writes the new value. There is no version check between the read and the write. Under concurrent access — which occurs on payday weekends when customers with multiple active plans have multiple installments processed within minutes of each other — the updates race. The last writer wins. The earlier write is silently lost. The audit log records each operation independently, so its sum is correct; the database value drifts away from the sum. The maximum observed concurrent updates against a single available_credit value is eight.

The Deadlock Storm

Operations alerts log a deadlock event approximately once every twenty minutes during business hours. The application logs the deadlock as an exception and immediately retries the failed transaction. The retry, in many cases, encounters the same deadlock. Under heavy load, the deadlock-retry pattern amplifies: each deadlock produces additional pressure on the locks, which produces additional deadlocks. The operations team has a runbook entry for this: when deadlock alerts exceed a threshold, restart the application connection pool. This breaks the cycle by terminating all in-flight transactions. The procedure takes between three and four minutes during which service is degraded.

The source code reveals the cause. Two code paths take locks on the same set of tables in opposite orders. The order placement flow takes a lock on the relevant Inventory row first, then takes a lock on the OrderItems row. The order cancellation flow takes a lock on the OrderItems row first, then on the Inventory row. When these two flows execute concurrently against the same order, they deadlock. One of the two code paths is in a logistics component licensed from a third-party vendor whose source code NovaMart does not have. The vendor was acquired six months ago and the original code has not been actively maintained since the acquisition.

The Ransomware Recovery

The ransomware event provides the most detailed evidence in the incident archive. The forensic report timeline is precise: the attacker gained service-user OS access on application server 7 at 14:23. By 14:31, they had read /etc/novamart/db.conf and connected to the database as the superuser whose credentials were stored there in plaintext. By 14:54, they had executed 340 SELECT queries against the Customers table, exfiltrating data in chunks. The encryption payload was deployed at 15:18. The Lahore data centre was shut down by the operations team at 15:47 in an attempt to limit the spread. By 17:12, restoration began. Restoration completed at 18:35, four hours and twelve minutes after the encryption payload was deployed.

The disaster recovery plan promises a 30-minute Recovery Time Objective. The actual recovery took more than eight times that. The reason is straightforward: the wal_level is set to minimal, which disables both logical decoding and point-in-time recovery between base backups. The last clean base backup was 18 hours old. The team had to choose between restoring 18 hours of work or losing everything since the last checkpoint, which was 28 minutes and 43 seconds before the corruption. They chose the checkpoint, losing the 28 minutes. The recovery procedure documented in the disaster recovery plan references pg_dump, which is not a point-in-time recovery tool; the team had to improvise. Improvisation under stress is what produced the four-hour timeline.

Part Seven: The Distributed Architecture in Practice

The case study has already noted that the five PostgreSQL instances do not replicate. What it has not yet detailed is how the system behaves day-to-day given this fact. The middleware layer, which is the only thing knitting the five instances into a coherent system, has accumulated workarounds. The workarounds have their own failure modes.

Fragmentation as Designed and as Used

The architects of the migration believed they were implementing horizontal fragmentation along principled lines. Orders was split into Orders_Online and Orders_Retail, on the theory that the two channels could be analysed independently and the split would allow each to be sized appropriately. In practice, 78 percent of the queries that touch Orders join both channels — every cross-channel revenue report, every fraud query that looks across channels, every customer activity view. Every such query is a cross-node operation.

Customers was split alphabetically by surname initial: customers with surnames beginning A through M reside on Node 2, N through Z on Node 3. The documentation describes this as 'range fragmentation on the customer dimension.' Analysis of the query log shows that 91 percent of queries against the Customers table do not filter by surname initial at all. They filter by customer identifier, by city, by purchase behaviour, by credit score range. The alphabetical fragmentation correlates with zero actual query access patterns; it makes every multi-customer query cross-node. This was not measured before the fragmentation was implemented.

The Middleware Join Pattern

When the middleware encounters a query that joins data across nodes, it does the following. It sends the relevant portion of the query to each node. It receives the result sets in Node.js memory. It performs the join in JavaScript. For a query that joins Orders_Online on Node 1 to Customers on Nodes 2 and 3, the middleware fetches up to 500,000 order rows from Node 1, up to 7 million customer rows from each of Nodes 2 and 3, and joins all of this in JavaScript. The worst observed case logged in the incident archive: a customer-orders join that loaded 14 million customer rows and 500,000 order rows into the Node.js heap, performed a nested loop join in JavaScript over the loaded arrays, took 87 seconds, and crashed the Node.js process with an out-of-memory error before producing a result. The query is in the source code under the comment 'TODO: this is slow, fix later.'

Order Placement and Its Discontents

Order placement on NovaMart Online is a multi-step operation: it must create an order record on Node 1, decrement inventory on Node 3, create a NovaPay plan on Node 4 if the payment method is installment (but actually on Node 1, given the connection-routing fix mentioned earlier), and create a logistics pickup record on Node 1. The four steps are sent by the middleware as four independent database writes. There is no protocol between them. If the first write succeeds but the second fails, the order exists but the inventory is not decremented. If the third write fails but the first two succeeded, the order exists with allocated inventory but no payment plan.

The reconciliation team's weekly report tracks these partial failures. There are approximately 340 per week. Of these, 47 percent are step-1-only — the order is created, no inventory is decremented, no NovaPay plan exists. 31 percent are steps 1 and 2 — the order is created and inventory is decremented, but no NovaPay plan or logistics pickup. 22 percent are complex partial states involving non-sequential failures. The reconciliation team handles each partial failure manually, by examining the operational logs and either completing the missing steps or rolling back the completed ones. Some require contacting the customer.

Part Eight: The NoSQL Layers

The migration plan called for a polyglot persistence architecture: PostgreSQL JSONB for flexible product attributes, Elasticsearch for product search, Redis for application-level caching. In principle, this is a reasonable design. In practice, all three layers were implemented without sufficient operational discipline, and all three are now in degraded states that affect customer-facing behaviour.

The JSONB Layer

During the migration, the team created a table called product_enrichment, intended to hold type-specific product attributes that did not fit the relational structure of the Products table. The schema was a single JSONB column called attributes. The intent was that each product category would have its own attribute structure — televisions would have screen_size, refrigerators would have capacity, gift cards would have face_value. The intent was not enforced. Four development teams wrote to this table from four different application paths without coordinating on attribute names, types, or structures.

Examination of the television category alone reveals seven distinct representations of the screen_size attribute across approximately 24,000 television product rows. Some rows store screen_size as a numeric value: 55. Some store it as a string: '55'. Some include the unit annotation: '55 inches'. Some nest it as an object: {'size': 55, 'unit': 'inches'}. Some wrap it in an array: [55]. Some have a key called screen_size with a null value. Some have no screen_size key at all and instead use a key called display_size. An aggregate query for mean television screen size by price bracket produces a numerical result. The result is wrong, because the AVG function coerces the string representations inconsistently and silently skips the nested objects and the arrays. The marketing team has been publishing this number monthly in newsletter communications. They are not aware that it is wrong.

The JSONB column has a GIN index. The index was created with the default jsonb_ops operator class. Analysis of the query log shows that 72 percent of queries against this column use containment operators — @> and <@. The jsonb_path_ops operator class produces smaller, faster indexes for these specific operators. The index has not been rebuilt with the appropriate operator class.

The Elasticsearch Layer

The Elasticsearch product index was populated through an outbox pattern. PostgreSQL triggers on the Products and product_enrichment tables wrote change events to an outbox table. A Python consumer process read events from the outbox and pushed them to Elasticsearch. The triggers are correct; they have continued to write events to the outbox throughout. The Python consumer crashed three months ago on a malformed product record — an unhandled UnicodeDecodeError raised on the description field of a product whose name contained an unsupported character. No alerting was configured on the consumer process. The crash went unnoticed.

The outbox table has accumulated 14.2 million unprocessed events. The Elasticsearch index contains prices, stock levels, and product descriptions that are between three and six months out of date depending on when each product was last modified. The customer-facing application queries Elasticsearch first; if Elasticsearch returns no results, the middleware falls back to a PostgreSQL full-text search. The fallback path is slow but functional, and it has been the primary search path for three months without anyone noticing.

During the most recent sale event — the 11.11 promotion — Elasticsearch returned pre-sale prices to approximately 1.2 million product search sessions. Customers added products to their carts at the pre-sale prices. NovaMart was contractually obligated under its terms and conditions to honour displayed prices. The revenue impact of honouring the stale prices was 18.4 million Pakistani rupees. The forensic review of this event noted, in passing, that a replication slot lag alert had been recommended during the original migration planning and never implemented.

The Redis Layer

Four independent application teams have written code that populates Redis. The teams did not coordinate. The resulting cache state has four key naming conventions, four serialisation formats, and four different TTL policies.

Product detail data is cached under keys of the form 'product:{id}' with a one-hour TTL and JSON serialisation. Product search results are cached under 'search:{query_hash}' with a 24-hour TTL and MessagePack serialisation. Store-specific promotions are cached under 'promo:{store_id}' with a 48-hour TTL and JSON. Inventory levels — the most-read cache type — are cached under 'inv:{store_id}:{product_id}' with no TTL. The inventory cache entries are permanent.

The permanent inventory cache entries mean that a product that goes out of stock continues to show as 'In Stock' to customers until the relevant Redis key is manually deleted. The operations team's runbook for inventory cache errors says, in its entirety, 'FLUSHDB.' Running FLUSHDB clears all four cache types simultaneously. The cache must then warm up again from scratch, with all reads going through to the database for three to four hours, during which the database typically becomes overloaded. The FLUSHDB procedure has been executed approximately twice per week for the past four months.

The Fraud Detection Graph

Fraud detection at NovaMart depends on identifying networks of customer accounts that share attributes — phone numbers, device fingerprints, CNICs, billing or delivery addresses. A group of three or more accounts that share two or more such attributes is a candidate fraud ring. The detection logic is implemented as a recursive Common Table Expression on the Customers table, joining the table to itself on shared attribute pairs and then transitively finding connected groups. It runs every hour on a scratch table without transaction isolation. A single run takes between 45 and 55 minutes. During the run, fraud detection is effectively offline; the credit scoring model that approves NovaPay applications cannot consult its output.

Three weeks ago, during a detection window, 340 synthetic accounts created NovaPay installment plans totalling 4.2 million Pakistani rupees. All 340 were approved by the credit scoring model. All 340 belonged to a single connected component in the customer similarity graph. The graph query would have identified them as a ring; the query was running at the time and its previous run was an hour stale. The fraud detection team identified the ring four hours later in a manual review and the plans were frozen, but the funds had already been disbursed to merchant accounts.

The graph itself — the customer similarity graph implicit in the Customers table — has not been characterised. Nobody at NovaMart has computed its node count, edge count, average degree, density, or the distribution of connected component sizes. This is the information needed to decide whether a recursive CTE approach is fundamentally appropriate for this workload or whether a different implementation (Apache AGE, an external graph database) is necessary.

Part Nine: The Warehouse and What It Reports

The MS SQL Server data warehouse was the second-largest component of the original migration plan, after the OLTP system. The ETL pipeline that populates it runs nightly. The SSAS multidimensional cube is consumed by analysts across the business; the board's quarterly report is built directly from cube queries. Eleven months ago, the cube began producing wrong numbers. The board does not know it has been receiving wrong numbers.

An external auditor was commissioned by the board's audit committee three months ago, after the CFO observed inconsistencies between the warehouse-reported revenue and the bank-reported cash receipts. The auditor's report, delivered four weeks ago, identified six categories of structural error in the warehouse. Each error has a different root cause.

Revenue and What It Excludes

The warehouse's revenue figure is computed in the ETL as SUM(OrderItems.unit_price * OrderItems.qty), aggregated by various dimensions. The computation does not join to the Refunds table. The Refunds table contains 1.2 million records representing partial and full reversals over the past three years. The computation also does not

exclude orders that were placed and then cancelled before fulfilment; cancellations are processed in a separate nightly job that runs after the ETL, so for one to three days each cancelled order continues to count as revenue. The computation also does not apply promotional discount codes from a separate Promotions table that the application uses to compute final prices; this table was never integrated into the ETL.

The combined effect is a revenue overstatement that the auditor measured at 8.4 percent over the eleven-month period in question — approximately PKR 340 million in phantom revenue. The board has been approving quarterly expansion targets, including authorisation for three new store openings, based on revenue figures 8 to 12 percent higher than reality.

Geographic Attribution

The warehouse's Customer_Dim implements a Type 1 Slowly Changing Dimension for customer attributes including city. When a customer updates their registered address, the city in Customer_Dim is overwritten with the new value. The historical city — the city the customer lived in at the time of past purchases — is not preserved. Across the customer base, 23 percent of customers have changed their registered city at least once over the past three years.

Every historical purchase by these customers is now attributed in the warehouse to their current city, not the city where they made the purchase. Regional revenue attribution is therefore systematically wrong for every city that has experienced net customer migration. The auditor estimated the misattribution as a 14 percent overstatement of Karachi revenue (which has received net migration) and a 19 percent understatement of Peshawar revenue (which has lost customers). The retail expansion team uses these regional revenue figures to decide where to open new stores; the three Q3 openings were chosen based on this misattributed data.

Date and Tax Filing

The Sales_Fact table records three dates per order: order_date, ship_date, and payment_received_date. The Time_Dim is joined to Sales_Fact on order_date only. The SSAS cube exposes a single Date dimension. Revenue by quarter is therefore computed against order_date.

Pakistani tax regulation under the Income Tax Ordinance requires revenue recognition on payment-received date for tax purposes. NovaMart's Q3 tax filing for the most recent fiscal year was prepared from the cube. The auditor's recomputation using payment_received_date instead of order_date produces a Q3 revenue figure that differs from the filed figure by 48 million Pakistani rupees. This is the difference between a profitable and an unprofitable quarter under the tax regulations. The audit notes this as a potential incorrect filing requiring amendment.

Basket Size and Operations

NovaMart Retail's point-of-sale system fragments single customer visits into multiple payment transactions for reasons related to payment terminal timeouts, split payment method handling, and recovery from network glitches. A single customer visit can produce three or four separate order records in the warehouse. The basket size metric — computed as total_revenue divided by order_count — treats each fragment as an independent basket. Retail basket size is understated in the warehouse by approximately 34 percent.

Store manager commissions are tied to basket size. The commission structure assumes a relationship between basket size and effective merchandising; managers whose stores produce higher baskets are paid more. The 34 percent understatement has been depressing commission calculations for 11 months. The auditor estimates underpayment in the range of PKR 12 million across the 134 store managers.

Inventory Turnover

The ETL runs at 2 AM. At 2 AM, retail stores are closed and no sales are in progress. The inventory snapshot taken by the ETL at this time reflects daily-maximum inventory — the state of stock before the next day's sales begin to deplete it. The warehouse's inventory turnover ratio uses this maximum as the denominator. The correct denominator for inventory turnover is average or minimum daily inventory.

The 2 AM snapshot overstates the denominator by approximately 40 percent relative to average daily inventory, which means the computed turnover ratio is understated by approximately 40 percent. The buying team uses

turnover figures to set reorder quantities and to negotiate vendor terms; systematic understatement has led the buying team to over-order. Warehouse storage costs have increased by 22 percent over the past year. The buying team attributes this to inflation in warehouse rental rates.

The NovaPay Default Rate

The NovaPay loan loss reserve provisioning is driven by the warehouse-reported default rate. The computation classifies a plan as defaulted at the moment any missed-payment threshold is crossed. The classification does not distinguish between plans that genuinely defaulted, plans that were restructured at the customer's request (payment amount adjusted, term extended) and are currently being paid under the restructured terms, and reversed installments that were collected and then refunded due to disputes. Restructured plans are counted as defaulted at restructuring time and then counted again when their restructured payments are made. Reversed installments are excluded from both numerator and denominator in a way that inflates the reported rate.

The auditor recomputed the default rate using a classification that distinguishes these categories. The actual default rate is 3.2 percent. The warehouse-reported rate is 7.8 percent. NovaPay has been provisioning loan loss reserves at the 7.8 percent rate, overstating reserves by approximately PKR 420 million. This capital is held against losses that are not occurring. It is not deployed in lending.

Part Ten: The Security Forensic Report

The external cybersecurity firm's forensic report on the ransomware event is 47 pages long. The most consequential section, for your purposes, is the section that enumerates the security failures identified during the investigation. The report identifies seven distinct failures. Each is a separate root cause. Each, the report concludes, would have been sufficient on its own to enable the attack; the fact that all seven existed simultaneously merely guaranteed that the attack would succeed once it was attempted.

Failure: Credential Management

All fifteen application servers connect to all five PostgreSQL instances using a single account named novamart_app. The account has SUPERUSER privileges. The password is identical across production, staging, development, and a test database accessible to junior developers. The password has not been rotated since the migration declared completion. It is stored in plaintext in a configuration file at /etc/novamart/db.conf on every application server. The file is owned by the novamart service user and is world-readable. The attacker, having obtained service-user OS context, read the file and connected as superuser within seconds.

Failure: Cardholder Data Retention

The Customers.card_last_sixteen column, despite its name, stores the full 16-digit primary account number for 2.3 million customers. The feature that populated this column — a 'save full card' option — was removed from the user interface in 2021. The data was never deleted. The forensic report confirms that all 2.3 million PANs were exfiltrated. The attacker pulled them in 340 SELECT queries spread across 23 minutes, deliberately paced to avoid triggering any rate-based alarm. There was no rate-based alarm; the pacing was unnecessary but the attacker did not know that. The PAN data falls under PCI DSS scope; NovaMart is now subject to Level 1 non-compliance proceedings.

Some fraction of the 2.3 million records may be within an 18-month mandatory retention window under Pakistani consumer credit law, which requires that payment dispute evidence be retained for 18 months following the disputed transaction. The exact fraction has not been determined. A blanket deletion of all 2.3 million records is therefore not legally permissible without first classifying each record.

Failure: Access Control Granularity

All application functions, regardless of their business purpose, use the same database role. A customer service agent querying for a single customer's data uses the same role as a NovaPay administrator querying for a complete account history. The role has read access to every customer-related table. The customer service application constructs queries of the form SELECT * FROM customers WHERE customer_id = $1. If the WHERE clause is removed — by a curious agent, by a compromised agent account, or by a malicious agent — the query returns all 14 million customer records. The reconstructed audit log (reconstructed from application logs, since the database itself had no audit log) records 14 incidents over the past year in which a customer service query returned more than 100 customer records in a single call. These were not investigated.

Failure: SQL Injection Surface

The penetration test commissioned after the ransomware event found 23 SQL injection points in the application code. The points are distributed across product search, customer lookup, order tracking, the store locator, and NovaPay account queries. All are caused by string concatenation: developers built query strings by appending input values directly. The most severe finding involves the order tracking endpoint, which accepts an order identifier from a URL parameter and concatenates it directly into a query. Using time-based blind injection on this endpoint, the penetration tester extracted the full Customers table schema in four requests, and subsequently demonstrated extraction of the contents of an administrative credentials table — including hashed passwords — in 23 requests at a rate of approximately two requests per minute. The extraction was not detected by any monitoring.

Failure: Audit Logging

The pg_audit extension was installed during the migration. It was never configured. The PostgreSQL audit log is empty. There is no record in the database of which queries were executed during the ransomware window. The forensic team's reconstruction of the attacker's activity relies on application-side logs and on a memory dump that was preserved before the affected systems were rebooted; the database itself has no contribution to the forensic record. The National Cyber Security Authority has issued a formal information demand under PECA Section 30 requesting a complete database access log covering the 72-hour window preceding the ransomware deployment. The company's response was that no such log exists. The regulator did not accept this as satisfactory.

Failure: Data at Rest

The PostgreSQL data directory sits on an ext4 filesystem without filesystem-level encryption. The ransomware operator read base table files directly from the filesystem before deploying the encryption payload. The forensic evidence is unambiguous: a memory dump recovered from the affected server contains the attacker's own exfiltration log, which records the paths of the base table files read (base/16384/* and similar). PostgreSQL's access controls — the role permissions, the row-level security policies that did not exist, the authentication configuration — are all irrelevant to a process that reads the underlying file system. There is no defence at the database layer against a process that can read /var/lib/postgresql directly.

Failure: Transport Encryption

The PostgreSQL configuration sets ssl = prefer rather than ssl = require. The 'prefer' setting attempts SSL but falls back to plaintext if SSL negotiation fails. Three application servers were discovered, during the post-incident network traffic analysis, to be connecting to the database in plaintext. The cause was an OpenSSL version incompatibility between the client libraries on those servers and the server's TLS 1.3 configuration; the servers attempted SSL, failed, and silently downgraded. The full query traffic from those three servers — including query

results containing customer data — was readable in plaintext on the data centre network. The forensic investigation confirms that the attacker captured at least some traffic before deploying the ransomware payload, though the volume of capture cannot be precisely measured.

The internal response to the discovery, two weeks before the ransomware event, was to add the three servers to a monitoring spreadsheet and schedule a fix. The fix was not implemented before the event.

Part Eleven: How the System Is Operated

Throughout this case study, the operations team's response to symptoms has been mentioned. It is worth assembling these references into a coherent picture, because the operational practices are themselves a contributor to the current state of the system, and any remediation plan must address them.

The Runbook

The operations team's standard operating procedures are documented in a single runbook of approximately 60 pages, written by the previous team during the migration and never updated. The runbook documents the response to thirteen distinct alert types. The documented response, in every case, is one of three actions: restart the application connection pool, restart Redis with FLUSHDB, or escalate to engineering. There is no diagnostic procedure documented for any of the alerts; the response is the same regardless of the alert's actual cause.

Performance Tuning

There is no record in the configuration history of any deliberate performance tuning. The PostgreSQL configuration is the post-installation default. The one documented configuration change is the disabling of autovacuum on four tables eighteen months ago. The change was made based on observation of correlation between autovacuum activity and a performance incident; the developer who made the change did not establish causation. The change has remained in place; the underlying cause of the original incident has never been identified; the side effect of the change — accumulating dead tuples — has worsened steadily ever since.

The Connection Pool Pathology

Fifteen application servers each maintain a connection pool of fifty connections. The aggregate demand is 750 connections against a max_connections of 100. The pool blocks waiting for connections to become available. Under sustained load, the block time at the pool level can exceed the application's HTTP request timeout, producing 503 errors to end users. The operations response, documented in the runbook, is to restart application servers until the queue clears. The restart procedure takes 8 to 12 minutes and degrades service during the window.

A PgBouncer configuration exists in the repository. The team that set it up did not complete the deployment. The configuration is present and could in principle be activated. There is no documentation of why the deployment was not completed; the email thread between the engineer who configured it and their manager indicates that they were pulled onto a different fire and never returned to the PgBouncer work. The application uses named prepared statements in the product search path and PostgreSQL advisory locks in the inventory reservation path. The team that designed the application was not aware that PgBouncer in transaction-pooling mode is incompatible with both of these features; the PgBouncer configuration in the repository is set to transaction-pooling mode.

Backup and Recovery

A base backup runs once per 24 hours, at 3 AM. The backups are written to a network-attached storage volume in the Islamabad data centre. The backups are not tested. The most recent attempt to verify a backup, conducted as part of the ransomware response, found that backups dated within the past two weeks were complete and restorable. Backups older than two weeks have not been tested and their state is unknown.

WAL archiving is not configured. The wal_level is set to minimal, which disables logical decoding and prevents point-in-time recovery between base backups. The disaster recovery plan promises a 30-minute Recovery Time Objective. The actual recovery during the ransomware event took four hours and twelve minutes. The disaster recovery plan has never been tested. No drill has ever been conducted.

Part Twelve: Your Engagement

You are engaged as the emergency consulting team. The board meets in 31 days. The regulator's deadline for the remediation report is 45 days; the board wants its own briefing before the regulator sees the response. You have 30 days. You are eight people. The expectations follow.

What the Board Wants

The board's expectations, communicated to you through the company's interim CTO, are operational stability, demonstrable security remediation, and accurate reporting. Operational stability means that the system can sustain the Friday evening load without crashing — the load that has caused crashes in eleven of the past sixteen Fridays. Demonstrable security remediation means that the regulator's specific findings are addressed and that there is technical evidence — code, configurations, audit logs — to demonstrate that the addresses are real, not paper assurances. Accurate reporting means that the warehouse produces numbers that match the bank statements and the audit trail; the board has lost confidence in the warehouse and will not trust it until they see independent verification.

The board does not have a checklist. They have engaged you because they do not know what specifically is wrong. They want a comprehensive remediation. They will be guided in the board meeting by the interim CTO, who will summarise your work. Your decision log — the document that records what you chose to fix, what you chose not to fix, and why — will be the basis of the CTO's summary. If your decision log is shallow, the CTO will sound shallow when summarising it, and the board will respond accordingly.

What the Regulator Wants

The NCSA's information demand is more specific. The demand is for technical evidence of remediation against seven findings (which are not enumerated in this document — they are in the forensic report and you must read it). The evidence must be technical: code, configurations, audit log samples, test results. Narrative descriptions of what would have happened with proper controls are not acceptable. The demand also requires explanation of the conditions under which the original failures occurred — root cause analysis, not just remediation.

What You Have to Deliver

Your deliverable to the company is a GitHub repository, accessible to the interim CTO and the regulator, containing all of the following:

- Setup scripts that recreate the broken initial state for any aspect of the system you have addressed. A clean PostgreSQL 16 installation, after running your setup, should reproduce the failure modes you have documented. This serves as the evidence-of-diagnosis component of the regulator's demand.

- Implementation artefacts for everything you have remediated. Migration scripts, stored procedures, application code patches, configuration files. Anything that exists only as a description and not as a runnable artefact will be treated as not having been done.
- Test scripts that verify your remediations are in place and functioning. A single command must be able to run the entire test suite and produce a machine-readable pass/fail output for every assertion. The evaluator will run this command.
- Benchmarks for any performance-related remediation. The benchmark data must be in raw form (CSV files), not summarised. The statistical analysis must be a script that runs against the raw data, not a paragraph of conclusions. The evaluator may re-run the benchmarks.
- Integration scripts that demonstrate the system behaves correctly under realistic load. The Friday evening peak — 500 concurrent product searches, 200 concurrent checkout flows, 150 concurrent order tracking queries, 100 concurrent NovaPay plan creations, 50 concurrent fraud detection queries, all running simultaneously for fifteen sustained minutes — must be reproducible from your scripts. A nightly maintenance window — ETL, search index reconciliation, autovacuum, WAL verification, fraud detection — must complete within the 2 AM to 6 AM window and must be idempotent (running it twice in succession must produce identical results). A scripted adversarial scenario, replaying the conditions of the ransomware attack, must show that the conditions no longer produce the original outcomes.
- Monitoring infrastructure, deployable with a single docker-compose command, that collects the metrics needed to detect future degradation of the system. The metrics you choose to collect are themselves a decision the evaluator will probe. Why these metrics and not others? Defend the choice.
- Decision logs, one per major problem area you addressed. The format is fixed: a quantitative diagnosis (numbers, not narrative; how did you determine the problem existed), the options you considered (with citations to the research literature where the options are described), the option you chose (with justification), the trade-offs you accepted in choosing it, and the conditions under which you would have chosen differently. The decision logs determine 20 percent of the evaluation score independently of any implementation.
- Security artefacts: the classification report for PAN data, the documentation of the key management approach for any encryption you implemented, the configuration of the audit logging, the TLS certificates and their generation procedure. These are what the regulator will examine.

Approach to AI Tooling

AI tooling is welcome and expected. There is no expectation that any portion of the implementation will be hand-written without AI assistance. There is also no leniency for misunderstanding what your AI tools produced. If your team submits a SERIALIZABLE isolation solution and cannot explain how PostgreSQL's serializable snapshot isolation implementation differs from a textbook 2PL serializable implementation, the evaluator will treat the submission as not understood, regardless of whether the implementation works. The viva is designed to detect this case in under five minutes per topic. There are no points for the visible artefact in the absence of understanding.

Conversely, there is no penalty for using AI tooling to accelerate the parts of the work that are routine. Writing the migration script that converts the Products schema, parsing the slow query log, generating the test harness — all of this is appropriate work for an AI tool. Choosing which schema to migrate to, deciding what to test, deciding which research paper's recommendation to follow when two papers disagree — these are decisions the team must make and defend.

Reference Catalogue

The following research works are relevant to the decisions you will need to make. They are presented as a single catalogue, not associated with specific problems — part of your work is to determine which works apply to which decisions. The evaluator has read all of these. A submission that cites them without engagement will be readily distinguishable from one that uses them.

Schema, Data Quality, and Migration

- Baazizi, M.A. et al. (2019). Efficient Discovery of Emerging Design Patterns in Large JSON Document Collections. VLDB 2019.
- Baazizi, M.A. et al. (2020). Parametric Schema Inference for Massive JSON Datasets. VLDB 2020.
- Pham, T. et al. (2021). JEDI: These Are Not the JSON Equivalences You Are Looking For. SIGMOD 2021.
- Liu, J. et al. (2022). SEED: Simple, Efficient, and Effective Data Management for Machine Learning. SIGMOD 2022.
- Ordonez, C. et al. (2018). Evaluating Hierarchical Queries in a Relational Database System. IEEE TKDE 2018.
- Qian, K. et al. (2020). Data Cleaning for Relational Databases: A Survey. ACM SIGMOD 2020.
- Sauer, C. et al. (2021). DiffStream: Differential Output Conformance Checking for Database Migration. VLDB 2021.
- Christophides, V. et al. (2021). An Overview of End-to-End Entity Resolution for Big Data. ACM Computing Surveys 2021.

Query Processing and Optimization

- Marcus, R. et al. (2019). Neo: A Learned Query Optimizer. VLDB 2019.
- Marcus, R. et al. (2021). Bao: Making Learned Query Optimization Practical. SIGMOD 2021.
- Neumann, T. & Radke, B. (2021). Improving Cardinality Estimation with Statistical Summaries of Base Table Predicates. VLDB 2021.
- Fan, J. et al. (2019). Incremental View Maintenance with Triple Lock Optimization Benefits. SIGMOD 2019.
- Nikolic, M. et al. (2018). Incremental View Maintenance with Conjunctive Queries. SIGMOD 2018.
- Kul, G. et al. (2018). Similarity Metrics for SQL Query Clustering. IEEE TKDE 2018.
- Tatbul, N. et al. (2018). Precision and Recall for Time Series. NeurIPS 2018.
- Leis, V. et al. (2018). Query Optimization Through the Looking Glass, and What We Found Running the Join Order Benchmark. VLDB 2018.
- Abebe, M. et al. (2022). Morphosys: Automatic Physical Design Metamorphosis for Distributed Database Systems. VLDB 2022.

Concurrency, Isolation, and Recovery

- Cahill, M.J., Röhm, U. & Fekete, A. (TODS 2009). Serializable Isolation for Snapshot Databases. ACM TODS 2009.
- Bernstein, P.A. et al. (2020). Optimistic Concurrency Control Revisited. ICDE 2020.
- Wu, Y. et al. (2021). BCC: Reducing False Aborts in OCC with Low Cost for In-Memory Databases. VLDB 2021.
- Zhang, Y. et al. (2021). Aria: A Fast and Practical Deterministic OLTP Database. VLDB 2021.

- Antonopoulos, P. et al. (2019). Socrates: The New SQL Server in the Cloud. SIGMOD 2019.
- Mohan, C. et al. (1992). ARIES: A Transaction Recovery Method. ACM TODS 1992.
- Raasveldt, M. & Muehleisen, H. (2020). Data Management for Data Science. CIDR 2020.

Distributed Systems and Replication

- Whittaker, M. et al. (2020). Scaling Replicated State Machines with Compartmentalization. VLDB 2020.
- Ousterhout, J. et al. (2021). SLOG: Serializable, Low-latency, Geo-replicated Transactions. VLDB 2021.
- Wolff, J. et al. (2020). Towards Declarative Distributed Database Queries. EDBT 2020.
- Garcia-Molina, H. & Salem, K. (1987). Sagas. SIGMOD 1987.
- Richardson, C. (2018). Microservices Patterns. Manning.
- Ceri, S. et al. (1983). Horizontal Data Partitioning in Database Design. SIGMOD 1983.

NoSQL, Graph Databases, and Change Data Capture

- Bonifati, A. et al. (2021). Graph Queries: From Theory to Practice. VLDB 2021 Tutorial.
- Then, M. et al. (2022). Graph Database Workloads: A Comparison of Native Graph Databases and Relational Databases. SIGMOD 2022.
- Besta, M. et al. (2019). High-Performance Graph Processing with FPGA. IPDPS 2019.
- Li, Z. et al. (2020). STROBE: Supporting Continuous Multi-Granularity Analysis Over Arbitrary Streams. SIGMOD 2020.
- Lu, J. et al. (2019). CacheLib: A Caching Engine for Web-Scale Services. OSDI 2019.
- Iorgulescu, C. et al. (2020). PolarDB Serverless. SIGMOD 2020.
- Red Hat / Debezium Project (2022). Debezium Documentation: CDC from PostgreSQL via Logical Replication. debezium.io.

Data Warehousing, ETL, and Analytics

- Schelter, S. et al. (2020). Automating Large-Scale Data Quality Verification. VLDB 2020.
- Vassiliadis, P. et al. (2021). Data Warehouse Maintenance in the Era of Agile Business Intelligence. JDIQ 2021.
- Hai, R. et al. (2019). Collaborative Data Acquisition. SIGMOD 2019.
- Gupta, A. et al. (2022). Efficient Query Processing over Heterogeneous Data Sources with the Qex System. SIGMOD 2022.
- Kimball, R. & Ross, M. (2013). The Data Warehouse Toolkit, 3rd ed. Wiley.

Security and Privacy

- Shastri, S. et al. (2019). The Seven Sins of Personal-Data Processing Systems Under GDPR. USENIX HotCloud 2019.
- Cao, Y. et al. (2021). Privacy-Preserving Database Operations for GDPR-Compliant Data Processing. SIGMOD 2021.
- Bellare, M. et al. (2020). Format-Preserving Encryption: FPE Schemes for Finite Languages. EUROCRYPT 2020.
- NIST SP 800-38G Rev. 1 (2019). Recommendation for Block Cipher Modes of Operation: Methods for Format-Preserving Encryption.
- Tan, R. et al. (2020). Fine-Grained Data Access Control with SQL. SIGMOD 2020.
- Sadeghian, A. et al. (2021). A Taxonomy of SQL Injection Attacks and Countermeasures. IEEE Security & Privacy 2021.

- Ferretti, L. et al. (2020). Blockchain-Based Tamper-Proof Database Audit Logging. IEEE TCC 2020.

Performance Engineering and Monitoring

- Van Aken, D. et al. (2021). An Inquiry into Machine Learning-Based Automatic Configuration Tuning Services on Real-World Database Management Systems. VLDB 2021.
- Li, G. et al. (2022). HUNTER: An Online Cloud Database Hybrid Tuning System for Throughput and Latency. SIGMOD 2022.
- Brewer, E. et al. (2021). Managing the MVCC Dead Tuple Problem at Scale. VLDB 2021 Industry Track.
- Dong, B. et al. (2020). Adaptive HTAP through Elastic Resource Scheduling. SIGMOD 2020.
- Ma, L. et al. (2022). AutoObvs: Automatic Observability for Database Query Performance Analysis. SIGMOD 2022.
- Gregg, B. (2020). Systems Performance: Enterprise and Cloud, 2nd ed. Prentice Hall.